

Exploration numérique de la fonction zêta de Riemann, primalité et IA

Auteur : hprzeta — Date : 26 mai 2026

Objectif : identifier les meilleures compétences, institutions, méthodes numériques, environnements Linux/Ubuntu et outils d'IA utiles pour une recherche expérimentale autour de $\zeta(s)$, des zéros, des L-fonctions et de la primalité.

1. Synthèse exécutive

L'exploration numérique sérieuse de l'hypothèse de Riemann n'est pas une simple programmation de $\zeta(s)$. Elle combine analyse complexe, théorie analytique des nombres, L-fonctions, calcul haute précision, arithmétique d'intervalles, FFT, calcul distribué/HPC, bases de données mathématiques et vérification formelle. Les chaînes les plus fiables sont reproductibles : C/C++ ou Sage/Python + FLINT/Arb/PARI/MPFR/GMP + tests unitaires + méthode de Turing + bases LMFDB/Odlyzko + éventuellement Lean/Mathlib pour formaliser les énoncés.

Le socle numérique le plus crédible est illustré par les travaux de Platt et la base LMFDB : 103 800 788 359 zéros de $\zeta(s)$ sont listés jusqu'à la hauteur 30 610 046 000, avec précision $\pm 2^{-102}$ et contrôle de complétude par une version rigoureuse de la méthode de Turing [turn1search1, turn1search5, turn1search6, turn1search22].

Pour coder et accélérer, l'écosystème prioritaire est Linux/Ubuntu, C/C++, Python/SageMath, PARI/GP, FLINT/Arb, GMP, MPFR, FFTW/OpenMP/MPI, et éventuellement CUDA/NVIDIA HPC SDK pour les parties parallélisables ; FLINT est une bibliothèque C libre pour théorie des nombres, factorisation, primalité, FFT, fonctions spéciales et arithmétique à boules avec bornes d'erreur [turn1search13, turn1search16, turn1search17].

Côté IA, aucune solution publique connue n'est spécialisée exclusivement dans la preuve de l'hypothèse de Riemann. Les meilleurs outils pertinents sont plutôt : AlphaProof/Gemini/Lean chez Google DeepMind pour raisonnement formel, les travaux OpenAI de preuve Lean, Meta/FAIR HyperTree Proof Search/Evariste, LeanDojo/LeanCopilot, plus des assistants de codage généraux pour produire, tester et auditer le code [turn1search7, turn1search11, turn1search38, turn1search32, turn1search40, turn1search49].

2. Classement des pôles de compétences à suivre

Priorité	Pôle / institution	Compétences distinctives	Méthodes clés	Pourquoi c'est utile
1	University of Bristol / ACRC / équipe Booker-Platt	Zéros de $\zeta(s)$, calcul rigoureux, L-fonctions, HPC.	Algorithme fenêtré de Booker/Platt, DFT/FFT, arithmétique d'intervalles, méthode de Turing.	Référence opérationnelle pour vérification numérique massive : Platt isole 103,8 milliards de zéros jusqu'à $3,0610046 \times 10^{10}$ [turn1search22].
2	LMFDB + réseau Sage/NSF/EP SRC/Simons	Base de données L-fonctions, zéros, modular forms, accès web et Sage.	Indexation, précision certifiée, vérification croisée, données interrogeables.	Source de données immédiatement exploitable pour benchmark et validation [turn1search1, turn1search5, turn1search6].
3	FLINT/Arb — Fredrik Johansson et communauté	Arithmétique exacte/approchée à bornes rigoureuses, C haute performance.	Ball arithmetic, fonctions spéciales, zeta_zeros.c, Keiper-Li, parallélisme.	Brique logicielle centrale pour calcul fiable de $\zeta(s)$, zéros, intégrales et polynômes [turn1search13, turn1search4].
4	Université de Bordeaux / Institut de Mathématiques de Bordeaux / PARI	Calcul formel de théorie des nombres et L-fonctions.	PARI/GP, gp2c, algèbre de nombres, factorisation, elliptic curves, modular forms, L-fonctions.	Très utile pour prototypage rapide et calculs arithmétiques ; PARI est conçu pour la théorie des nombres [turn1search55, turn1search71].
5	University of Minnesota / Odlyzko	Statistiques des zéros, GUE, calculs à très grande hauteur.	Odlyzko-Schönhage, FFT, évaluation multiple de $\zeta(s)$, tables de zéros.	Méthode fondamentale pour blocs de valeurs et tests statistiques des espacements [turn1search23, turn1search20, turn1search21].
6	University of Bristol — Keating/Snaith/Conrey	Random Matrix Theory, moments de ζ , lien physique quantique-nombres premiers.	RMT/GUE, moments, modèles de matrices aléatoires.	Indispensable si l'axe inclut statistiques fines et modèles spectraux [turn1search26, turn1search30, turn1search29].
7	Lean / Microsoft Research / Lean FRO / Mathlib	Formalisation mathématique et preuves vérifiées.	Lean 4, Mathlib, preuve machine-checkable, intégration IA.	Voie majeure pour transformer des résultats numériques ou conjecturaux en énoncés vérifiables [turn1search49, turn1search51, turn1search52, turn1search67].

Priorité	Pôle / institution	Compétences distinctives	Méthodes clés	Pourquoi c'est utile
8	Google DeepMind	IA de raisonnement mathématique formel.	AlphaProof, AlphaGeometry 2, Gemini, apprentissage par renforcement, Lean.	A atteint niveau médaille d'argent IMO 2024 ; AlphaProof a résolu aussi un problème de théorie des nombres [turn1search7, turn1search11].
9	Meta AI / FAIR + IP Paris	Recherche de preuves neuronale.	HyperTree Proof Search, online training, Metamath, Lean miniF2F.	Approche AlphaZero-like intéressante pour automatisation de preuves [turn1search32, turn1search33, turn1search36].
10	Tsinghua / BMSA / YMSC et CAS/USTC	L-fonctions, zeta multiple, méthodes analytiques, liens quantiques.	Séminaires number theory, L-fonctions, Riemann hypothesis ; expérience trapped-ion sur zéros de Riemann.	Pôle chinois actif : BMSA cite analytic methods for L-functions and RH ; USTC/CAS a mesuré 80 zéros via ingénierie Floquet [turn1search61, turn1search64, turn1search73].

3. Méthodes mathématiques et numériques incontournables

Méthode	Rôle dans $\zeta(s)$ /RH	Compétence à acquérir	Outils typiques
Euler-Maclaurin	Évaluation générale de $\zeta(s)$, bonne maîtrise des restes mais moins efficace à grande hauteur.	Analyse asymptotique, bornes d'erreur, nombres de Bernoulli.	PARI/GP, mpmath, Arb/FLINT [turn1search23].
Riemann-Siegel / Hardy Z(t)	Évaluation efficace sur la ligne critique $\text{Re}(s)=1/2$; détection de changements de signe.	Implémenter Z(t), $\theta(t)$, restes de Gabcke, Newton/bissection.	C/C++, Arb zeta_zeros, mpmath, Sage [turn1search4, turn1search22].
Odlyzko-Schönhage	Évaluation multiple rapide de $\zeta(s)$ sur des grilles denses ; accélère les campagnes massives.	FFT, séries de Dirichlet, rational function evaluation, amortissement mémoire/temps.	FFTW, C/C++, MPI/OpenMP [turn1search23].
Booker/Platt fenêtre	Version rigoureuse et efficace pour isoler beaucoup de zéros avec précision certifiée.	DFT/Poisson summation, Gaussian windows, interval arithmetic, Turing method.	C, MPFI/MPFR/GMP, clusters HPC [turn1search22].
Méthode de Turing	Contrôle de complétude : s'assurer qu'aucun zéro n'est manqué dans un intervalle.	Calcul de N(T), argument principe, bornes de S(t).	Implémentations maison, Arb, scripts de vérification [turn1search6, turn1search22].
Arithmétique d'intervalles / ball arithmetic	Transformer les résultats numériques en certificats avec bornes d'erreur.	Propagation d'erreur, précision adaptative, arrondis dirigés.	Arb/FLINT, MPFR, MPFI [turn1search13, turn1search4, turn1search22].
Critères Keiper-Li	RH équivaut à la positivité de certains coefficients λ_n ; axe expérimental alternatif.	Calcul de coefficients, transformées binomiales, haute précision.	Arb examples keiper_li [turn1search4].
Random Matrix Theory / GUE	Modèles statistiques des espacements de zéros et moments de $\zeta(s)$.	Probabilités, matrices aléatoires, physique quantique chaotique.	Python/Sage, Julia, R, simulations [turn1search26, turn1search30].
Formalisation Lean	Énoncés et preuves vérifiés : zeta, L-fonctions, Dirichlet, RH formelle.	Lean 4, Mathlib, analyse complexe formalisée.	Lean, VS Code, mathlib, LeanDojo [turn1search49, turn1search67, turn1search68].

4. Environnement Linux/Ubuntu recommandé

Les travaux de calcul haute précision autour de $\zeta(s)$ sont naturellement proches de Linux : compilation C/C++, contrôle fin des bibliothèques numériques, scripts reproductibles, HPC, conteneurs et ordonnancement de jobs. Platt a utilisé du C, de l'arithmétique d'intervalles MPFI, une précision de travail de 300 bits, et jusqu'à 32 nœuds du cluster Bluecrystal II de Bristol [turn1search22]. Arb/FLINT documente des exemples compilables en C, multi-threadés, incluant zeta_zeros.c, real_roots.c et keiper_li.c [turn1search4].

Couche	Choix recommandé	Fonction
OS	Ubuntu LTS / Debian / Rocky Linux en cluster	Stabilité, paquets scientifiques, compatibilité HPC, scripts shell reproductibles.
Langages	C/C++ pour noyau ; Python/Sage pour expérimentation ; GP/PARI pour calculs rapides ; Lean pour preuves.	Séparer performance, prototypage et vérification.
Bibliothèques numériques	GMP, MPFR, MPFI, FLINT/Arb, FFTW, OpenBLAS, mpmath.	Précision arbitraire, intervalles, FFT, algèbre rapide [turn1search13, turn1search16].

Couche	Choix recommandé	Fonction
Calcul distribué	OpenMP, MPI, Slurm, Snakemake/Make, Docker/Singularity.	Paralléliser les fenêtres de t, garder les logs et checksums.
GPU	CUDA/NVIDIA HPC SDK uniquement pour parties adaptées : FFT massives, scans vectorisés, ML.	Le SDK NVIDIA supporte C/C++/Fortran, OpenACC/CUDA, profiling/debug sous Linux [turn1search17].
Qualité	Tests unitaires, property-based testing, CI GitHub/GitLab, double implémentation indépendante.	Éviter les faux zéros, erreurs d'arrondi, bugs de parsing ou de précision.

5. IA utiles : ce qu’elles font vraiment

Point essentiel : une IA générative ne doit jamais être considérée comme preuve d’un résultat sur RH. Elle peut accélérer la programmation, proposer des lemmes, traduire en Lean, générer des tests, relire du code C/Python, optimiser un pipeline et chercher des preuves formelles ; la validation finale doit rester numérique certifiée ou formelle.

IA / projet	Organisation	Utilité concrète pour $\zeta(s)$ /RH	Limite
AlphaProof + Gemini + Lean	Google DeepMind	Raisonnement formel, génération/recherche de preuves dans Lean, apprentissage par renforcement ; a atteint 28/42 à l'IMO 2024 avec AlphaGeometry 2 [turn1search7, turn1search11].	Pas un outil public spécialisé RH ; dépend de formalisation préalable.
OpenAI neural theorem prover	OpenAI	Recherche de preuves Lean ; miniF2F, olympiades, boucle où les preuves trouvées enrichissent l'entraînement [turn1search38].	Travail de recherche ; pas garanti pour analyse complexe avancée.
HyperTree Proof Search / Evariste	Meta AI / FAIR	Recherche de preuves avec online training, Metamath et Lean miniF2F [turn1search32, turn1search36].	Code public annoncé comme artefact non prêt à l'emploi [turn1search36].
LeanDojo / LeanCopilot	Caltech + communauté	RAG, interaction Lean, preuve assistée, correction et exploration formelle [turn1search40].	Nécessite savoir Lean/mathlib.
GitHub Copilot / ChatGPT / Gemini / Claude / Code Llama	Microsoft/OpenAI, Google, Anthropic, Meta	Coder prototypes, tests, scripts Slurm, notebooks, documentation, refactoring.	Peut halluciner ; toujours compiler, tester et comparer à LMFDB/Arb.
Wolfram/Mathematica, Magma, Maple + assistants	Éditeurs CAS	Vérification symbolique, calculs de référence, expérimentation.	Souvent propriétaire ; moins reproductible que pile open-source.

6. Institutions et angles de recherche par zone

6.1 Amérique / UA-US

Priorités : University of Minnesota/Odlyzko pour grands calculs et statistiques des zéros ; Institute for Advanced Study/Princeton pour théorie analytique et percées conceptuelles ; LMFDB/SageMath comme infrastructure ouverte ; Caltech/LeanDojo pour IA+preuve formelle. Odlyzko–Schönhage reste central pour l’évaluation multiple rapide de $\zeta(s)$ et s’applique aussi aux L-fonctions et séries de Dirichlet [turn1search23]. SageMath fédère NumPy, SciPy, FLINT, PARI/GP, Maxima, GAP, R et d’autres paquets dans une interface Python utile en recherche [turn1search43, turn1search44].

6.2 Europe

Priorités : Bristol pour zéros numériques rigoureux et RMT ; Bordeaux pour PARI/GP et L-fonctions computationnelles ; Bayreuth/UniDistance/ETH autour de la formalisation de zeta et L-fonctions en Lean ; universités de Cambridge/Edinburgh/Oxford pour IA mathématique et théorie des nombres. Le projet de formalisation Lean de zeta et L-fonctions ajoute dans Mathlib la définition de riemannZeta, l’équation fonctionnelle, le produit d’Euler et une formalisation de l’énoncé RH [turn1search67, turn1search68].

6.3 Chine

Priorités : Tsinghua-BIMSA-YMSC pour L-fonctions, zeta multiple, méthodes analytiques et RH ; CAS/AMSS pour mathématiques fondamentales ; USTC/CAS pour l’axe Hilbert–Pólya/physique quantique. BIMSA décrit explicitement des axes “Arithmetic of L-functions” et “Analytic methods for L-functions and automorphic forms” incluant Riemann hypothesis [turn1search61]. La CAS rapporte une expérience USTC mesurant les 80 premiers zéros via un qubit ion piégé et l’ingénierie de Floquet, utile pour l’angle physique/quantique plutôt que pour une preuve directe [turn1search73].

7. Plan d'action recommandé sur 12 mois

Phase	Objectif	Livrables
Mois 1-2	Installer et maîtriser la pile Ubuntu.	Ubuntu LTS, gcc/clang, GMP/MPFR, FLINT, SageMath, PARI/GP, Lean 4, VS Code ; notebook de premiers zéros comparé à LMFDB.
Mois 3-4	Reproduire les calculs de base.	Implémenter Hardy $Z(t)$, Riemann-Siegel, bisection/Newton ; comparer les 10^3 premiers zéros à Arb/LMFDB.
Mois 5-6	Passer à la rigueur numérique.	Arithmétique d'intervalles, bornes d'erreur, tests de non-régression, logs de précision.
Mois 7-8	Apprendre Turing method et contrôles de complétude.	Détecter zéros sur intervalles, prouver qu'aucun zéro n'est oublié.
Mois 9-10	Optimiser et paralléliser.	Fenêtres indépendantes, OpenMP/MPI, Slurm, profiling, reproductibilité.
Mois 11-12	Ajouter IA et formalisation.	Lean : formaliser définitions/lemmes simples ; IA pour générer tests, audit, documentation et proof sketches.

8. Compétences prioritaires à développer

- Analyse complexe : prolongement analytique, équation fonctionnelle, principe de l'argument, intégrales de contour.
- Théorie analytique des nombres : $\zeta(s)$, L-fonctions de Dirichlet, formules explicites, $N(T)$, $\pi(x)$, $\psi(x)$.
- Algorithmes numériques : Euler-Maclaurin, Riemann-Siegel, Odlyzko-Schönhage, FFT, interpolation/sampling.
- Calcul certifié : MPFR/GMP, arithmétique d'intervalles, ball arithmetic, tests de précision adaptative.
- Programmation scientifique : C/C++, Python/Sage, PARI/GP, Linux, Make/CMake, profiling, CI.
- HPC : OpenMP/MPI, Slurm, conteneurs, stratégie de partitionnement par fenêtres de hauteur t .
- IA et preuve formelle : Lean 4, Mathlib, LeanDojo/LeanCopilot, usage prudent de LLM pour code et preuve.
- Data engineering mathématique : LMFDB, formats de tables, checksums, métadonnées, reproductibilité.

9. Architecture de projet conseillée

Une architecture saine sépare : (1) notebooks d'exploration, (2) noyau C/C++ certifié, (3) scripts de batch HPC, (4) base de résultats, (5) tests de comparaison LMFDB/Arb, (6) module Lean pour formaliser les énoncés et hypothèses. Le dépôt doit contenir un fichier CITATION, un environnement reproductible, des logs de version de compilateur et des checksums de sorties.

Dossier	Contenu
/notebooks	Explorations Sage/Python, visualisations, comparaisons LMFDB.
/src	C/C++ : Hardy Z , Riemann-Siegel, intervalles, FFT, wrappers FLINT.
/tests	Tests unitaires, régression contre zéros connus, tests de précision.
/hpc	Scripts Slurm, paramètres de fenêtres, logs et reprises après échec.
/lean	Énoncés Lean, dépendance Mathlib, expérimentations LeanDojo.
/data	Petits jeux de données versionnés ; gros fichiers externes avec hash.
/docs	Rapports, méthodologie, limitations, sources bibliographiques.

10. Bibliographie et sources Web utilisées

- LMFDB — Zeros of $\zeta(s)$ — <https://www.lmfdb.org/zeros/zeta/> [turn1search1]
- LMFDB — Completeness — <https://www.lmfdb.org/zeros/zeta/Completeness> [turn1search5]
- LMFDB — Reliability — <https://www.lmfdb.org/zeros/zeta/Reliability> [turn1search6]
- D. J. Platt, Isolating some non-trivial zeros of Zeta — https://research-information.bris.ac.uk/files/78836669/platt_zeta_submitted.pdf [turn1search22]

- Odlyzko & Schönhage, Fast algorithms for multiple evaluations — <https://www.ams.org/journals/tran/1988-309-02/S0002-9947-1988-0961614-2/S0002-9947-1988-0961614-2.pdf> [turn1search23]
- Arb examples — zeta_zeros, keiper_li — <https://arblib.org/examples.html> [turn1search4]
- FLINT — Fast Library for Number Theory — <https://flintlib.org/> [turn1search13]
- PARI/GP — <https://pari.math.u-bordeaux.fr/> [turn1search55]
- SageMath — <https://www.sagemath.org/> [turn1search43]
- University of Bristol — Riemann hypothesis and quantum physics/RMT — <https://www.bristol.ac.uk/math/research/highlights/riemann-hypothesis/> [turn1search26]
- Google DeepMind — AlphaProof/AlphaGeometry 2 — <https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/> [turn1search7]
- OpenAI — formal math Lean — <https://openai.com/index/formal-math/> [turn1search38]
- Meta/FAIR — HyperTree Proof Search — <https://arxiv.org/abs/2205.11491> [turn1search32]
- Lean Mathlib — <https://lean-lang.org/use-cases/mathlib/> [turn1search49]
- Formalizing zeta and L-functions in Lean — <https://arxiv.org/html/2503.00959v2> [turn1search67]
- Tsinghua/BIMSA Number Theory — https://qzc.tsinghua.edu.cn/en/academics/yjfx/BIMSA/Number_Theory.htm [turn1search61]
- Chinese Academy of Sciences — trapped-ion Riemann zeros — https://english.cas.cn/research/highlight/qp/202108/t20210810_277469.shtml [turn1search73]
- NVIDIA HPC SDK — <https://developer.nvidia.com/hpc-sdk> [turn1search17]

Note finale : ce rapport recommande une approche expérimentale rigoureuse. Les résultats numériques, même massifs, ne constituent pas une preuve de RH ; ils servent à tester, conjecturer, calibrer et soutenir des arguments, avec bornes d'erreur et reproductibilité.